Introdução ao Assembly para Arquitetura x86 16 bits

Cláudio Luís V. Oliveira
claudio.oliveira@fatec.sp.gov.br

Uma das principais características da linguagem Assembly consiste no acesso aos registradores da CPU, desta forma, como primeiro passo é importante conhecer os registradores utilizados nos processadores Intel/AMD para os computadores pessoais. A CPU possui alguns registradores internos, entre eles AX, BX, CX e DX que são registradores de uso geral. Estes registradores podem ser referenciados modo completo, por exemplo, AX ou também parcialmente, neste caso, devemos referenciá-los como, por exemplo, AH e AL. Abaixo estão relacionados os registradores:

Registrador	Função	Registrador	Função
AX	Acumulador	BX	Base
CX	Contador	DX	Dados
DS	Segmento de Dados	ES	Segmento Extra
SS	Segmento de Pilha	CS	Segmento de Código
BP	Apontador da Base	SI	Índice Fonte
DI	Índice Destino	SP	Apontador de Pilha
IP	Apontador da Próxima Instrução	F	Flag

Programa DEBUG

O debug é um pequeno utilitário da linha de comando que permite a criação e análise de pequenos programas na linguagem Assembly. Ele pode apenas criar arquivos com a extensão .com, e por causa das características deste tipo de programa, eles não podem exceder os 64 Kb e devem iniciar sempre no endereço 0100h, dentro do segmento específico de memória. Os principais comandos do programa debug são:

Comando	Descrição
A	Montar instruções simbólicas em código de máquina
D	Mostrar o conteúdo de uma área da memória
E	Entrar dados na memória, iniciando num endereço específico
G	Rodar um programa executável na memória
N	Dar nome a um programa
P	Proceder, ou executar um conjunto de instruções relacionadas
Q	Sair do programa debug
R	Mostrar o conteúdo de um ou mais registradores
T	Executar passo a passo as instruções
U	Desmontar o código de máquina em instruções simbólicas
W	Gravar um programa em disco

Também é possível visualizar os valores dos registradores internos da CPU usando o debug. Para iniciá-lo, basta digitar debug na linha de comando:

```
debug
```

É possível utilizar o programa DEBUG através do emulador PCE - PC Emulator (http://www.hampa.ch/pce/).

Para criar um programa no debug, será usado inicialmente o comando "a" (assemble) e, em seguida, o endereço de início do programa no segmento de memória. Posteriormente, devem ser especificados os comandos desejados, por exemplo:

```
a 100
MOV AX, 0002
MOV BX, 0004
ADD AX, BX
nop
```

Após digitar o último comando do programa, pressionar a tecla enter duas vezes para sair do modo de edição. A execução da aplicação criada pode ser realizada passo-a-passo através do comando "t", onde também é possível visualizar o estado dos registradores após cada instrução realizada.

Em qualquer momento é possível visualizar e alterar os valores dos registradores através do comando "r". Uma alternativa consiste em especificar o registrador desejado dentro do comando "r", por exemplo:

```
r AX
```

Neste caso, após mostrar o valor do registrador AX, o programa também permite a alteração do seu valor. No prompt mostrado digita-se o novo valor ou apenas tecle enter para manter o valor atual, por exemplo:

```
r AX
AX 0002
:
```

Operações Aritméticas

ADD destino, fonte: adiciona o operador fonte ao destino.

SUB destino, fonte: subtrai o operador fonte do destino.

MUL fonte: realiza uma multiplicação entre o conteúdo do acumulador AL ou AX pelo operando-fonte, devolvendo o resultado no acumulador AX caso a operação tenha envolvido AL com um operando de 8 bits, ou em DX e AX caso a operação tenha envolvido AX e um operando de 16 bits.

DIV fonte: Se o divisor é de 8 bits, o registrador AX de 16 bits é tomado como dividendo e se o divisor é de 16 bits, o par de registradores DX:AX será tomado como dividendo, tomando a palavra alta de DX e a baixa de AX. Se o divisor for um byte, então o quociente será armazenado no registrador AL e o resto em AH. Se for uma palavra, então o quociente é armazenado em AX e o resto em DX.

Interrupções

O próximo exemplo irá mostrar uma pequena aplicação que permite repetir a exibição da cadeia de caracteres "Olá pessoal":

```
- a 100
0C1B:0100 JMP 125   ;Salta para o endereço 125h
0C1B:0102
- e 102  'Ola pessoal!' 0d 0a '$'
- a 125
0C1B:0125 MOV DX,0102    ;Copia o endereço da string para DX
0C1B:0128 MOV AH,09      ;Copia o valor 09 para AH
0C1B:012A INT 21         ;Mostra a string
0C1B:012C INT 20         ;Finaliza o programa
```

O exemplo a seguir irá mostrar uma pequena aplicação que permite repetir a exibição da cadeia de caracteres “Olá pessoal” conforme o valor armazenado no registrador CX:

```
- a 100
0C1B:0100 JMP 125   ;Salta para o endereço 125h
0C1B:0102
- e 102  'Ola pessoal!' 0d 0a '$'
- a 125
0C1B:0125 MOV DX,0102    ;Copia o endereço da string para DX
0C1B:0128 MOV CX,0001    ;Quantas vezes a string será mostrada
0C1B:012B MOV AH,09      ;Copia o valor 09 para AH
0C1B:012D INT 21         ;Mostra a string
0C1B:012F DEC CX         ;Subtrai 1 de CX
0C1B:0130 JCXZ 0134      ;Se CX é igual a 0 salta para o endereço 0134
0C1B:0132 JMP 012D       ;Salta ao endereço 012D
0C1B:0134 INT 20         ;Finaliza o programa
```

Criando aplicações com o NASM

O NASM é um montador gratuito para a linguagem Assembly, com ele é possível realizar o processo de criação de aplicações. Digite o exemplo a seguir em um editor de texto e salve o arquivo com o nome olapessoal.asm:

```
ORG 100h
    mov dx,msg
    mov ah,9
    int 21h
    mov ah,4Ch
    int 21h
    msg db 'Ola pessoal! (Assembly)',0Dh,0Ah,'$'
```

Em seguida utilizamos o montador para criar um programa executável:

```
nasm olapessoal.asm -o olapessoal.com
```